Binning and Binarization

Turning a continuous column into intervals

Contents

Binning and Binarization 2
32.1 Why replace a number with an interval? 2
32.2 Equal-width binning 3
32.3 Equal-frequency binning 4
32.4 K-means binning 5
32.5 Binarization 5
32.6 The lecture's Titanic experiment 6
32.7 The Scikit-learn API in detail 6
32.8 Choosing a representation 7
32.9 Recap, practice and sources 7

CHAPTER 32

Binning and Binarization

Turning a continuous column into intervals

WHAT THIS CHAPTER BUILDS

Discretization trades resolution for simplicity. This chapter compares equal-width, equal-frequency and k-means bins, derives what each method is actually optimising, and explains why the lecture's Titanic experiment barely moved the accuracy.

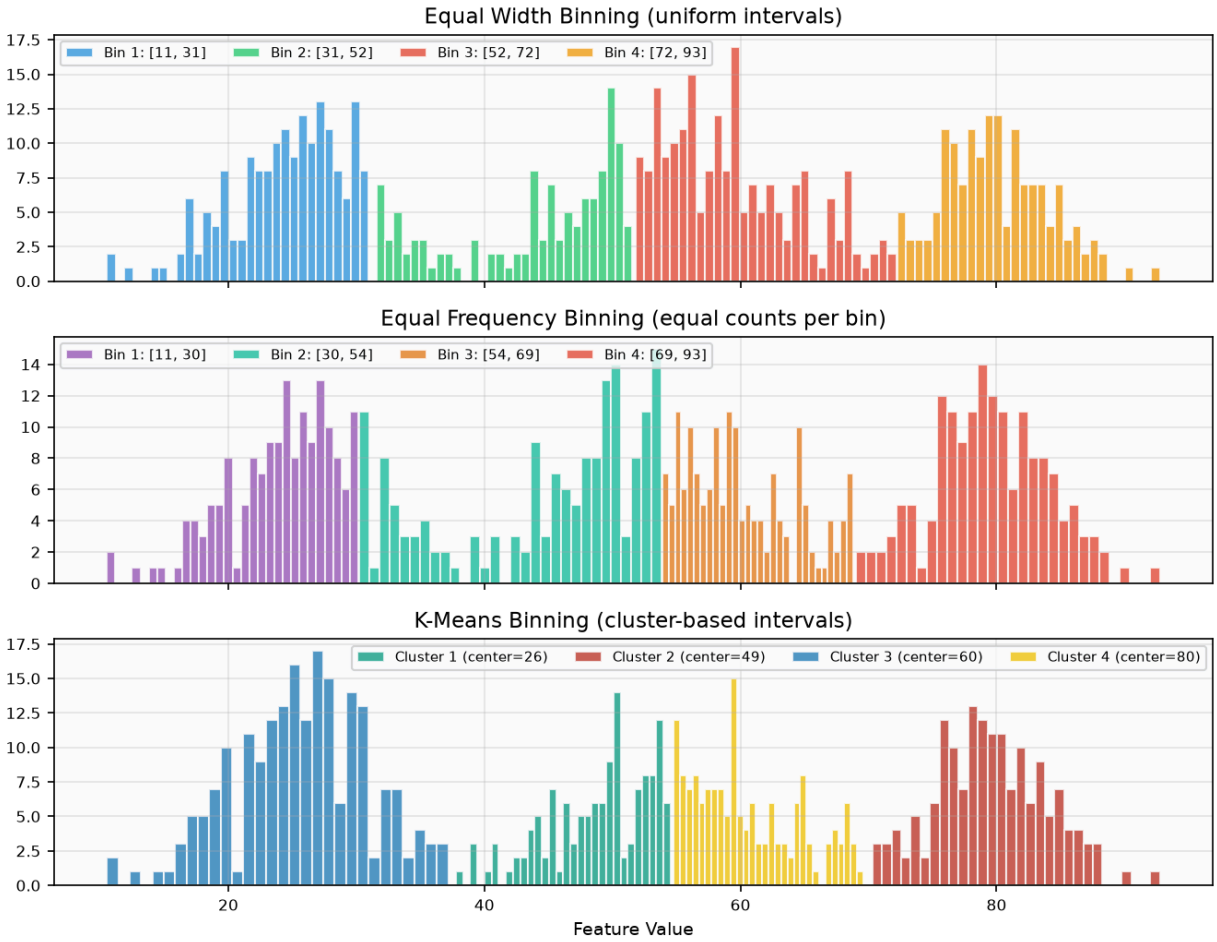
32.1 Why replace a number with an interval?

A continuous feature can enter a model in three ways. It can stay continuous, it can be split into intervals, or it can be reduced to a binary flag. Binning is the middle option: each value is replaced by the index of the interval that contains it.

The motivation is not that intervals are more accurate. It is that a linear model can only represent a straight-line effect. If the probability of an event rises until age 30, flattens until 50, and then falls, a single coefficient on age cannot express that shape. A set of age bins can, because each bin receives its own coefficient. The same idea applies to a scorecard, where “income between 30k and 50k” is easier to audit than a coefficient on a continuous income.

Binning also reduces the influence of small fluctuations and extreme values. A single income of 10 million cannot dominate a bin the way it can dominate a mean or a distance. That is a real benefit for noisy measurements.

The cost is equally real. Once a value is replaced by a bin, all variation inside the bin is gone. Two applicants with incomes 31k and 49k become identical. The choice of cut points becomes a modelling decision with no unique correct answer. Binning can also create a step function where the true relationship is smooth, which increases bias.



Equal-width bins divide the range into intervals of the same length. Equal-frequency bins put approximately the same number of observations in each interval. K-means bins place cut points between clusters in one dimension.

> **Binning is a bias–variance trade**
>
> More bins mean less bias and more variance. Fewer bins mean more bias and less variance. There is no universally best number of bins; the right choice depends on the sample size, the shape of the relationship, and the model that consumes the bins.

32.2 Equal-width binning

Equal-width binning is the simplest rule. Given a feature with minimum a and maximum b, choose K bins and set the width to

$$w = \frac{b - a}{K}$$

The boundaries are

$$b_k = a + kw, \quad k = 0, 1, \ldots, K$$

A value x belongs to bin k when $b_k \le x < b_{k+1}$, with the last bin closed at the maximum. The implementation is a sequence of cut points and a search.

```
import numpy as np
import pandas as pd
```

```
from sklearn.preprocessing import KBinsDiscretizer

age = pd.DataFrame({"age": [18, 22, 25, 29, 31, 35, 42, 48, 55, 61, 70]})

ew = KBinsDiscretizer(
    n_bins=4,
    encode="ordinal",
    strategy="uniform",
)
age["age_bin"] = ew.fit_transform(age[["age"]]).ravel()
print(ew.bin_edges_)
```

The advantage is transparency: every bin has the same width, so the boundaries are easy to explain. The disadvantage is that a single extreme value stretches the range and makes the other bins narrow. If 99% of the data lie between 18 and 60 but one value is 200, most observations fall into the first bin and the remaining bins are almost empty. Equal width is therefore a poor default for skewed features.

`KBinsDiscretizer` clips values outside the training range to the nearest outer bin during `transform`. That is convenient in production, but it also means a new extreme value is silently treated like the largest training value. Record the learned edges and monitor for values outside them.

32.3 Equal-frequency binning

Equal-frequency binning, also called quantile binning, chooses boundaries so that each bin contains approximately the same number of observations. For K bins, the cut points are the empirical quantiles at $\frac{1}{K}, \frac{2}{K}, \ldots, \frac{K-1}{K}$:

$$p_k = \frac{k}{K}, \quad q_k = \text{quantile}(x, p_k)$$

The first bin contains values up to q_1, the second contains values between q_1 and q_2, and so on. If the feature is skewed, the bins become wider in the sparse tail and narrower in the dense centre. That is exactly the opposite of equal width, and it is usually what we want when the goal is to give each bin enough observations to estimate a stable effect.

```
ef = KBinsDiscretizer(
    n_bins=4,
    encode="ordinal",
    strategy="quantile",
)
age["age_bin_ef"] = ef.fit_transform(age[["age"]]).ravel()
print(ef.bin_edges_)
```

Equal-frequency binning has two practical complications. First, repeated values can make quantiles coincide, so the realised number of bins may be smaller than requested. A feature with many zeros is a common example. Second, quantiles are estimated from the training sample; a different sample can produce slightly different edges. Fit on training data, then apply the same edges to validation and test data.

> **Quantile is not a magic fairness fix**
>
> Equal-frequency bins give each interval the same number of rows, not the same meaning. A bin that contains the top 25% of incomes is not “the rich quartile” in any causal sense. Interpret the boundaries, not only the count.

32.4 K-means binning

K-means binning treats the one-dimensional feature as data to be clustered. It chooses K centres $\mu_1, \ldots, \mu_K$ and assigns each observation to the nearest centre, minimising

$$\sum_{k=1}^{K} \sum_{x_i \in C_k} (x_i - \mu_k)^2$$

The cut points are placed midway between adjacent sorted centres. In one dimension, k-means is a principled version of "put boundaries where the data are sparse": it spends more centres where the observations are dense.

```
km = KBinsDiscretizer(
    n_bins=4,
    encode="ordinal",
    strategy="kmeans",
    random_state=42,
)
age["age_bin_km"] = km.fit_transform(age[["age"]]).ravel()
print(km.bin_edges_)
```

The result depends on the initialisation and on the value of K. Scikit-learn runs a k-means algorithm internally and uses `random_state` to make the result reproducible. For large datasets, `subsample` limits the number of rows used to learn the centres; the default is 200,000. That is a computational convenience, not a statistical correction.

K-means bins are useful when the feature has natural clusters, such as a mixture of two age groups. They are less interpretable than quantiles because the boundaries are not round numbers. They also inherit k-means' sensitivity to the scaling of the feature, although in one dimension the scale only changes the units, not the ordering of distances.

Strategy	What is equal	Strength	Weakness
Uniform	Bin width	Simple, transparent	A single outlier can empty the other bins
Quantile	Approximate count per bin	Robust to skew, stable bin counts	Duplicate values can collapse bins; edges vary by sample
K-means	Within-bin variance is low	Boundaries follow density	Needs K and initialisation; less interpretable

32.5 Binarization

Binarization is the special case $K = 2$ with a chosen threshold τ:

$$x' = \begin{cases} 1 \text{ if } x > \tau \\ 0 \text{ otherwise} \end{cases}$$

Scikit-learn's `Binarizer` uses a strict greater-than comparison. A value exactly equal to the threshold becomes 0. That detail matters when the threshold is a meaningful boundary such as zero.

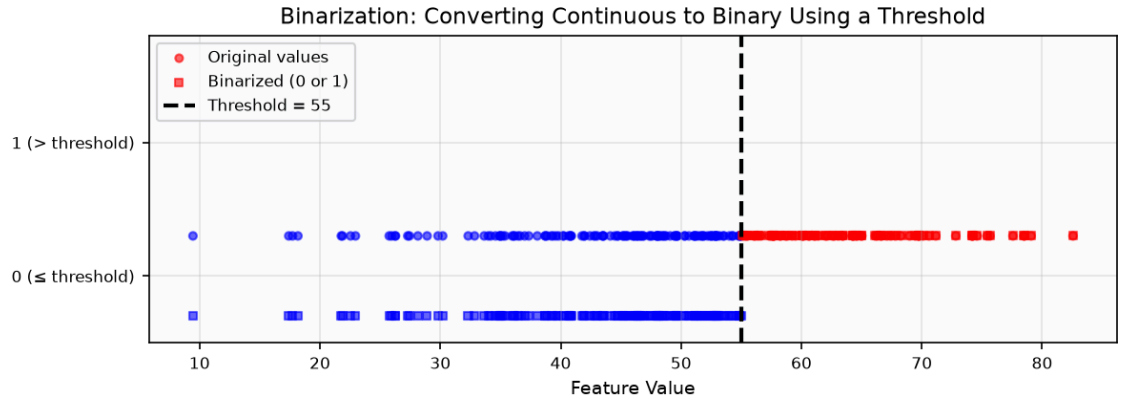
```
from sklearn.preprocessing import Binarizer

counts = pd.DataFrame({"word_count": [0, 0, 1, 3, 8, 20]})
```

```
binarizer = Binarizer(threshold=0)
counts["has_text"] = binarizer.fit_transform(counts[["word_count"]]).ravel()
print(counts)
```

Binarization is most useful for zero-inflated features: a word count of zero versus any positive count, a purchase amount of zero versus any purchase, or a sensor reading below a detection limit. It is also common in text classification, where the presence of a term is more stable than its raw frequency.

Binarization is not a substitute for binning when the feature has more than two meaningful regions. Collapsing age into “young” and “old” may be appropriate for a specific question, but it discards the middle. Choose the threshold from domain knowledge or a validation curve, not from the test set.



Binarization keeps only the side of the threshold. Values above tau become 1; values at or below tau become 0.

32.6 The lecture’s Titanic experiment

The lecture applies binning to Titanic features, especially age and fare. Age is continuous and has missing values, so it must be imputed before binning. Fare is strongly right-skewed, which makes equal-width bins a poor choice: most passengers pay little, a few pay a lot, and the expensive tail stretches the range.

The experiment compares several bin counts and all three strategies, then trains the same classifier on each version. The baseline accuracy is about 0.6290. The best binned version is about 0.6301. That is a difference of roughly one tenth of one percentage point, which is well within the noise of a small test set.

The honest conclusion is that binning did not help much here. There are several reasons. The relationship between age and survival is not a clean step function. The dataset is small, so a few changed predictions move the accuracy. The classifier may already be able to use the continuous feature, and binning may throw away information without adding enough non-linearity to compensate.

This is a useful result. A feature-engineering technique is not a performance button. It is a change to the representation, and its value must be measured against a baseline with the same split and the same model.

> **Do not choose bins on the test set**
>
> If you try ten bin counts and three strategies, then report the best test accuracy, you have used the test set to choose a hyperparameter. The reported score is optimistic. Use cross-validation on the training set to choose the binning scheme, then evaluate once on the held-out test set.

32.7 The Scikit-learn API in detail

`KBinsDiscretizer` has three important parameters:

- `n_bins`: the number of intervals. It can be an integer or an array-like with one entry per feature.
- `encode`: `"ordinal"` returns integer bin indices; `"onehot"` returns a sparse one-hot matrix; `"onehot-dense"` returns a dense one-hot matrix.
- `strategy`: `"uniform"`, `"quantile"`, or `"kmeans"`.

After fitting, `bin_edges_` is an array of arrays. Entry `i` contains the edges for feature `i`. The number of edges is usually `n_bins + 1`, but it can be smaller when quantiles coincide. `n_bins_` reports the realised number of bins per feature.

`transform` maps new values to bins using the learned edges. Values below the smallest edge or above the largest edge are assigned to the first or last bin. `inverse_transform` maps a bin index back to its centre, not to the original value. That distinction is important: binning is not lossless, and an inverse transform cannot recover the value that was inside the bin.

`Binarizer` has a simpler API: `threshold` and `copy`. It does not learn anything from the data, so it has no `fit` state beyond the threshold. It can still be placed in a pipeline for consistency.

32.8 Choosing a representation

Start with the continuous feature and a baseline model. If the model is linear and the relationship is visibly non-linear, try a small number of quantile bins. Quantile bins are usually the safest default because they guarantee roughly equal support. Use equal-width bins only when the scale is meaningful and the feature is not heavily skewed. Use k-means bins when the feature appears to have natural clusters and interpretability is secondary.

For tree-based models, binning is often unnecessary. A tree can split a continuous feature at any threshold, and it already represents a step function. Binning may still help by reducing the number of candidate splits and by smoothing noise, but the gain is not automatic. For linear models, one-hot encoding the bins gives each interval its own coefficient; ordinal encoding imposes an order and a single linear effect, which defeats much of the purpose unless the bins are genuinely ordered categories.

Handle missing values before binning. `KBinsDiscretizer` expects numeric input and does not treat `NaN` as a separate bin. A missing indicator can be added alongside the binned feature if missingness itself is informative.

32.9 Recap, practice and sources

The chain is: binning replaces a continuous value with an interval index; equal-width bins are simple but outlier-sensitive; equal-frequency bins balance support but can collapse on ties; k-means bins follow density but need a random seed; binarization is the two-bin case with a strict threshold; fit the bin edges on training data; evaluate against a continuous baseline before believing that discretization helped.

> **Lock-in line**
>
> Binning does not add information. It changes the shape a model is allowed to fit, and it charges for that flexibility in resolution.

Practice

1. Bin a skewed income column with equal width and equal frequency. Compare the number of observations in each bin.
2. Add one extreme value and repeat. Which strategy changes more?
3. Fit `KBinsDiscretizer(strategy="kmeans")` twice with different random seeds. Are the edges stable?
4. Binarize a count column at 0 and at its median. Explain when each threshold is appropriate.

5. Reproduce the Titanic comparison with a fixed cross-validation split. Is the difference larger than the fold-to-fold variation?

Sources and scope

Video 32, “Binning and Binarization”, CampusX 100 Days of Machine Learning. The lecture demonstrates equal-width, equal-frequency and k-means binning, binarization, and a Titanic accuracy comparison. The derivation of the k-means objective and the warning about selecting bins on the test set are supplementary explanations added here for completeness.